K.R. MANGALAM UNIVERSITY

School of engineering and technology

Lab Assignment -1:

Algorithm Foundations Instructions

Submitted by:

Satyam Singh

(2401420022)

Submitted to:

Dr. Aarti

Assignment Tasks

Task 1: Agorithm Growth Observation:

```python
import time
import pandas as pd
import matplotlib.pyplot as plt
```

```python
def O1(n):
    return 9999 * 9999

def On(n):
    s = 0
    for i in range(n):
        s += 1
    return s

def On2(n):
    s = 0
    for i in range(n):
        for j in range(n):
            s += 1
    return s

def Olog2(n):
    count = 0
    while n > 1:
        n //= 2
        count += 1
    return count
```

```python
def avg_time(func, n, repeat=5):
    total = 0
    for _ in range(repeat):
        start = time.perf_counter()
        func(n)
        total += time.perf_counter() - start
    return total / repeat
```

```python
sizes = [10, 100, 500, 1000]

functions = {
    "O(1)": O1,
    "O(n)": On,
    "O(n^2)": On2,
    "O(log n)": Olog2
}

results = {"n": sizes}
```

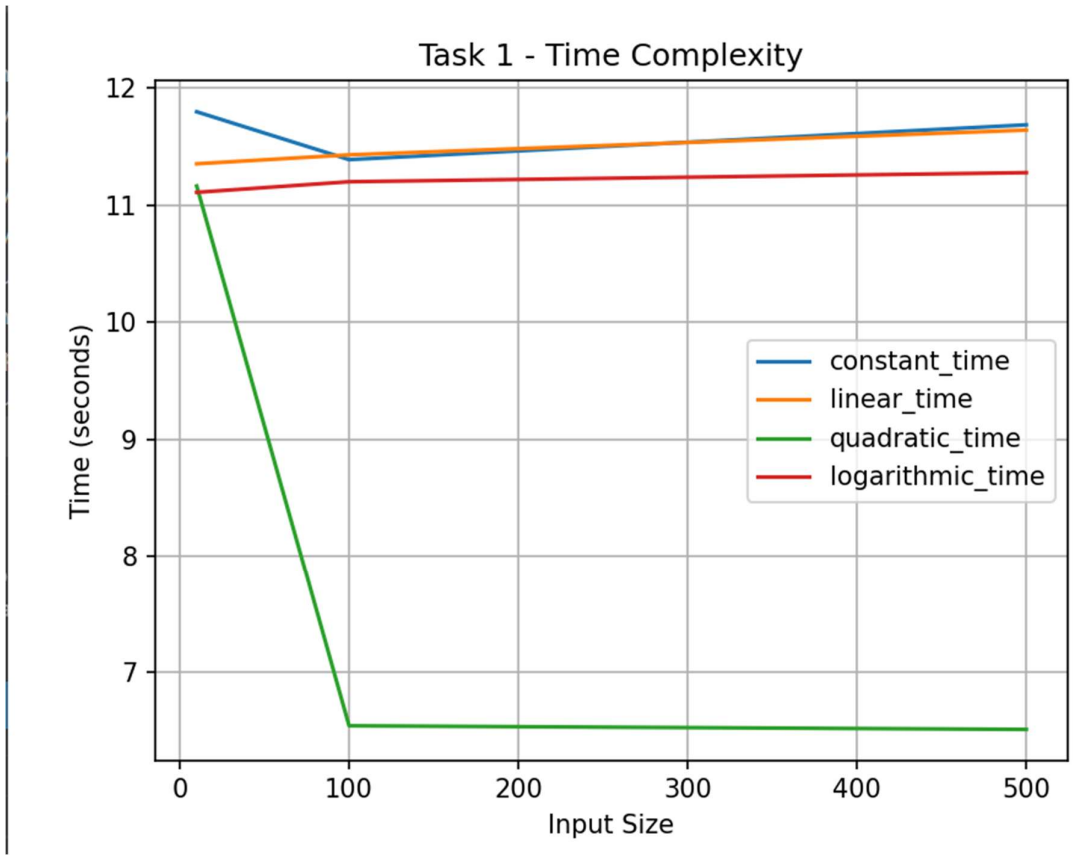
```python
for name, func in functions.items():
    results[name] = [avg_time(func, n) for n in sizes]
```

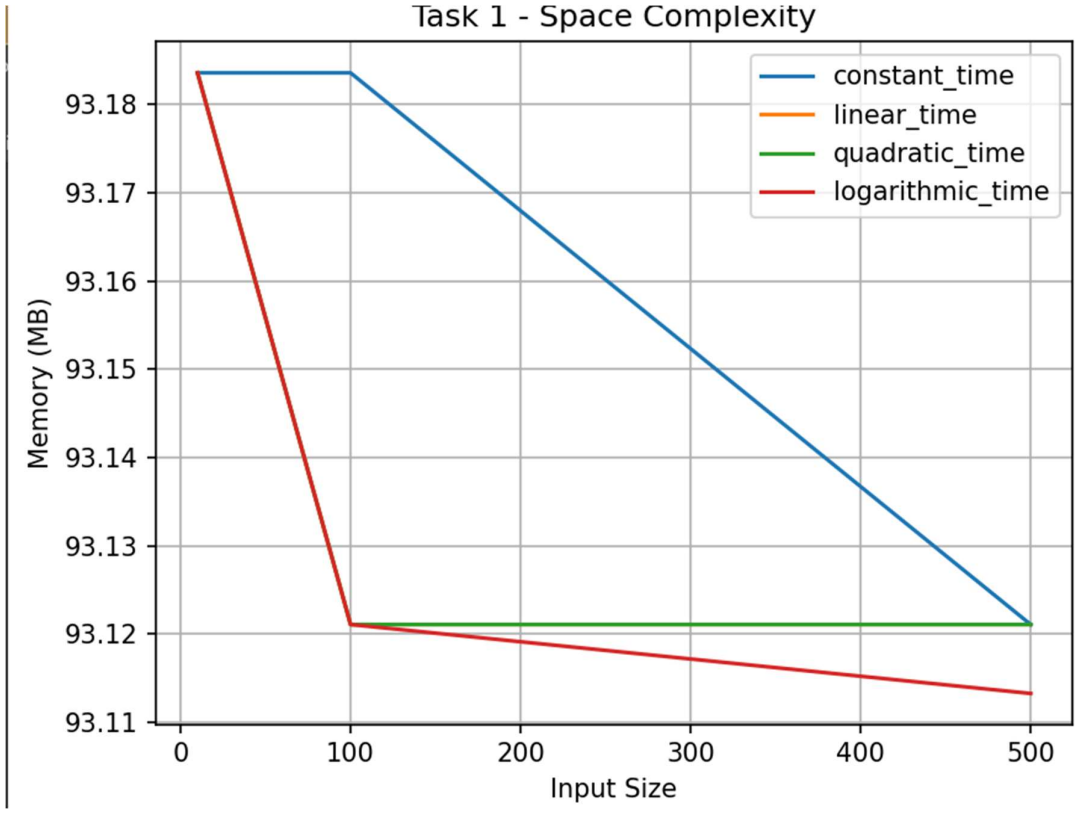
```python
df = pd.DataFrame(results)
print("\nExecution Time Table:\n")
print(df)
```

```
Execution Time Table:

     n      O(1)      O(n)   O(n^2)  O(log n)
0   10  0.000014  0.000002  0.000012  0.000001
1  100  0.000014  0.000007  0.000353  0.000001
2  500  5.600274e-07  0.000067  0.009250  0.000001
3 1000  5.600274e-07  0.000140  0.126193  0.000004
```

```python
for col in df.columns[1:]:
    plt.plot(df["n"], df[col], marker="o", label=col)

plt.xlabel("Input size (n)")
plt.ylabel("Execution time (seconds)")
plt.title("Input Size vs Execution Time")
plt.legend()
plt.grid(True)
plt.yscale("log")
plt.show()
```

Result:


Task 1 - Time Complexity
Time (seconds)
Input Size
12
11
10
9
8
7
0
100
200
300
400
500
constant_time
linear_time
quadratic_time
logarithmic_time


Task 1 - Space Complexity
constant_time
linear_time
quadratic_time
logarithmic_time
Memory (MB)
93.18
93.17
93.16
93.15
93.14
93.13
93.12
93.11
0
100
200
300
400
500
Input Size

Task 2: Best, Average, Worst Case Analysis:

```python
import time
import random
import pandas as pd
import matplotlib.pyplot as plt
```

```python
# Linear Search
def linear_search(arr, target):
    for i in arr:
        if i == target:
            return True
    return False

# Binary Search
def binary_search(arr, target):
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return True
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return False
```

```python
# Timing function
def avg_time(func, arr, target, repeat=5):
    total = 0
    for _ in range(repeat):
        start = time.perf_counter()
        func(arr, target)
        total += time.perf_counter() - start
    return total / repeat
```

```python
sizes = [100, 500, 1000, 5000, 10000]

results = []

for n in sizes:
    arr = random.sample(range(1, n * 10), n)

    arr_sorted = sorted(arr)

    best_target = arr[0]
    avg_target = arr[n // 2]
    worst_target = -1

    # Linear Search Timings
    lin_best = avg_time(linear_search, arr, best_target)
    lin_avg = avg_time(linear_search, arr, avg_target)
```

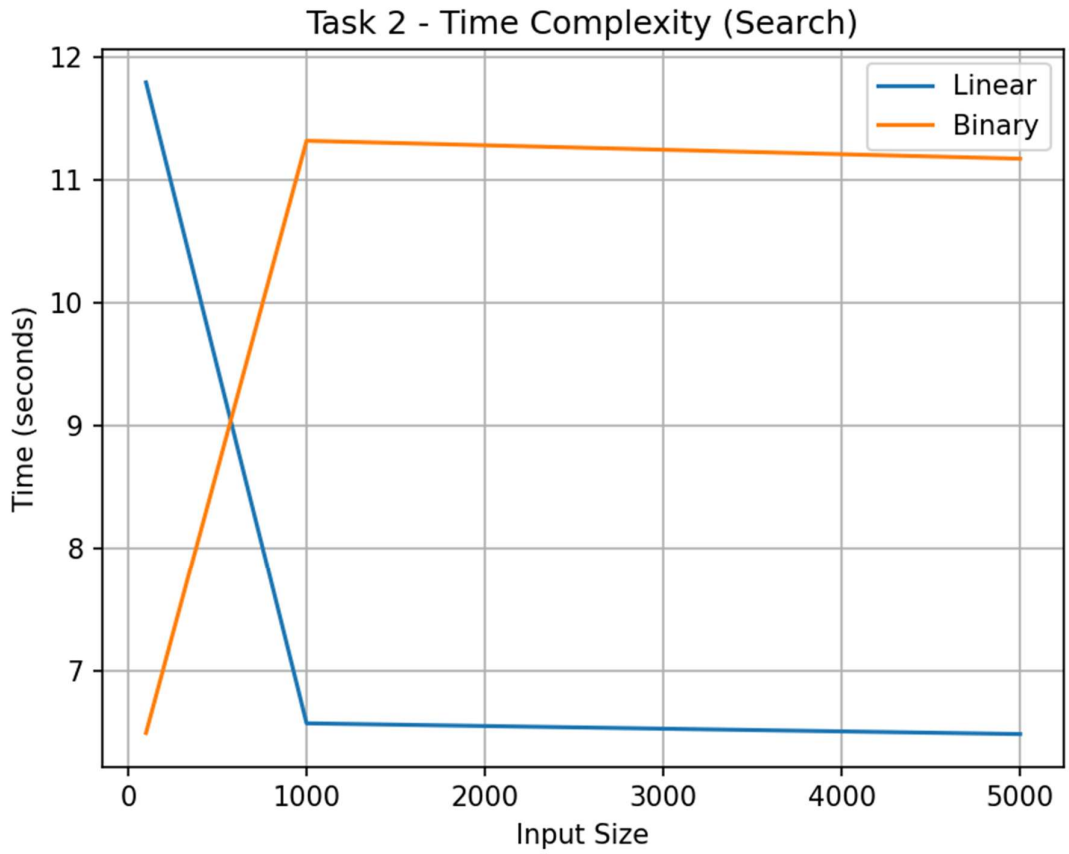
```python
    # Linear Search Timings
    lin_best = avg_time(linear_search, arr, best_target)
    lin_avg = avg_time(linear_search, arr, avg_target)
    lin_worst = avg_time(linear_search, arr, worst_target)

    # Binary Search Timings
    bin_best = avg_time(binary_search, arr_sorted, arr_sorted[n // 2])
    bin_avg = avg_time(binary_search, arr_sorted, avg_target)
    bin_worst = avg_time(binary_search, arr_sorted, worst_target)

    results.append([n, lin_best, lin_avg, lin_worst, bin_best, bin_avg, bin_worst])
```

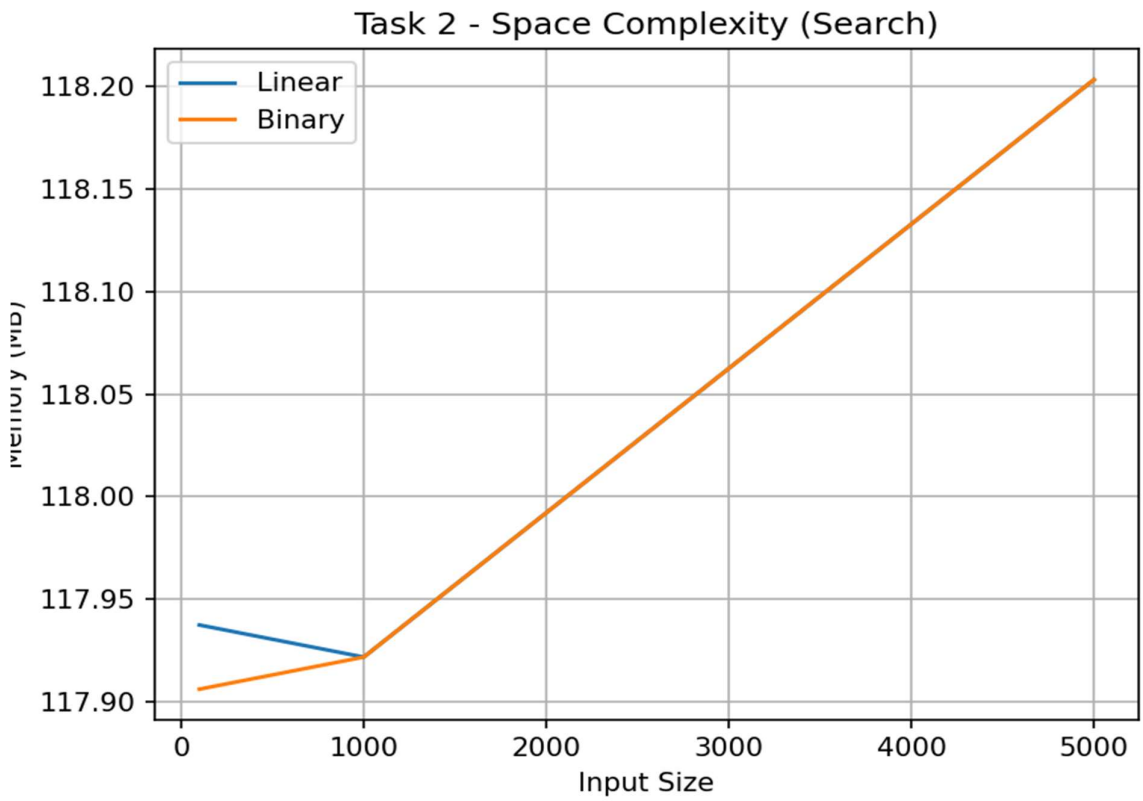
```python
df = pd.DataFrame(results, columns=[
    "n",
    "Linear Best", "Linear Avg", "Linear Worst",
    "Binary Best", "Binary Avg", "Binary Worst"
])

print("\nExecution Time Table:\n")
print(df)
```

```
Execution Time Table:

       n  Linear Best  Linear Avg  Linear Worst  Binary Best  Binary Avg  \
0    100  1.520812e-06    0.000005      0.000008     0.000004    0.000002
1    500  6.599152e-07    0.000016      0.000028     0.000004    0.000003
2   1000  8.000307e-07    0.000031      0.000073     0.000007    0.000005
3   5000  1.799967e-06    0.000163      0.000139     0.000009    0.000009
4  10000  1.839921e-06    0.000272      0.000686     0.000006    0.000004

   Binary Worst
0      0.000003
1      0.000003
2      0.000006
3      0.000007
4      0.000005
```

```python
plt.figure(figsize=(10, 6))

plt.plot(df["n"], df["Linear Best"], marker="o", label="Linear Best")
plt.plot(df["n"], df["Linear Avg"], marker="o", label="Linear Avg")
plt.plot(df["n"], df["Linear Worst"], marker="o", label="Linear Worst")

plt.plot(df["n"], df["Binary Best"], marker="o", label="Binary Best")
plt.plot(df["n"], df["Binary Avg"], marker="o", label="Binary Avg")
plt.plot(df["n"], df["Binary Worst"], marker="o", label="Binary Worst")

plt.xlabel("Input Size (n)")
plt.ylabel("Execution Time (seconds)")
plt.title("Linear Search vs Binary Search (Best, Avg, Worst Case)")
plt.legend()
plt.grid(True)

plt.yscale("log")
plt.show()
```

Result:


Task 2 - Time Complexity (Search)
Linear
Binary
Time (seconds)
Input Size
12
11
10
9
8
7
0
1000
2000
3000
4000
5000


Task 2 - Space Complexity (Search)
Linear
Binary
118.20
118.15
118.10
118.05
118.00
117.95
117.90
0
1000
2000
3000
4000
5000
Input Size

Task 3: Recursion and Recurrence Validation:

```python
import time
import pandas as pd
import matplotlib.pyplot as plt
```

```python
# Factorial (Recursive)
fact_calls = 0

def factorial(n):
    global fact_calls
    fact_calls += 1

    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

# Fibonacci (Naive Recursion)
fib_naive_calls = 0

def fib_naive(n):
    global fib_naive_calls
    fib_naive_calls += 1

    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

# Fibonacci (DP / Memoization)
fib_dp_calls = 0

def fib_dp(n, memo=None):
    global fib_dp_calls
    fib_dp_calls += 1

    if memo is None:
        memo = {}

    if n in memo:
        return memo[n]

    if n <= 1:
        return n

    memo[n] = fib_dp(n - 1, memo) + fib_dp(n - 2, memo)
    return memo[n]
```

```python
def run_and_measure(func, n):
    start = time.perf_counter()
    value = func(n)
    end = time.perf_counter()
    return value, (end - start)
```

```python
fact_inputs = [5, 10, 15, 20]
fib_inputs = [5, 10, 15, 20, 25, 30]

results = []
```

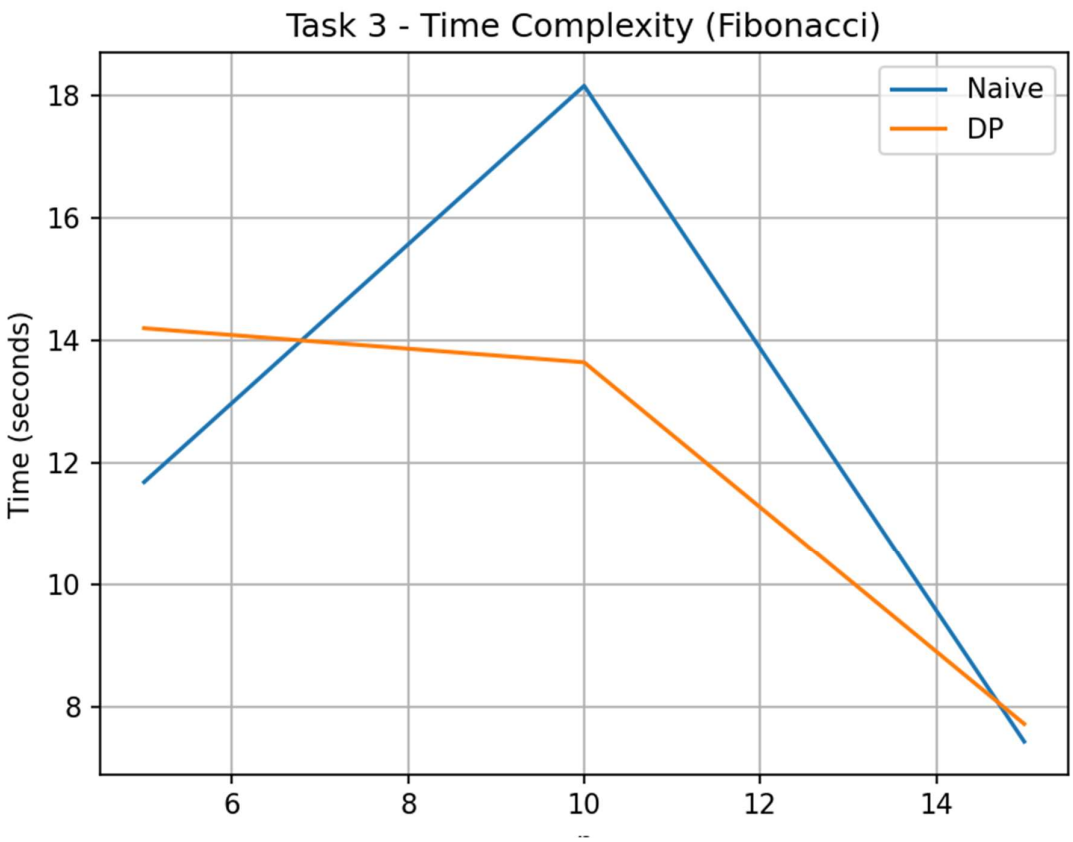
```python
fib_inputs = [5, 10, 15, 20, 25, 30]

results = []

# Factorial Results
for n in fact_inputs:
    fact_calls = 0
    val, t = run_and_measure(factorial, n)

    results.append({
        "Algorithm": "Factorial (Recursion)",
        "n": n,
        "Value": val,
        "Function Calls": fact_calls,
        "Time (sec)": t
    })

for n in fib_inputs:
    # Naive Fibonacci
    fib_naive_calls = 0
    val1, t1 = run_and_measure(fib_naive, n)

    results.append({
        "Algorithm": "Fibonacci (Naive)",
        "n": n,
        "Value": val1,
        "Function Calls": fib_naive_calls,
        "Time (sec)": t1
    })

    # DP Fibonacci
    fib_dp_calls = 0
    val2, t2 = run_and_measure(lambda x: fib_dp(x, {}), n)

    results.append({
        "Algorithm": "Fibonacci (DP)",
        "n": n,
        "Value": val2,
        "Function Calls": fib_dp_calls,
        "Time (sec)": t2
    })
```

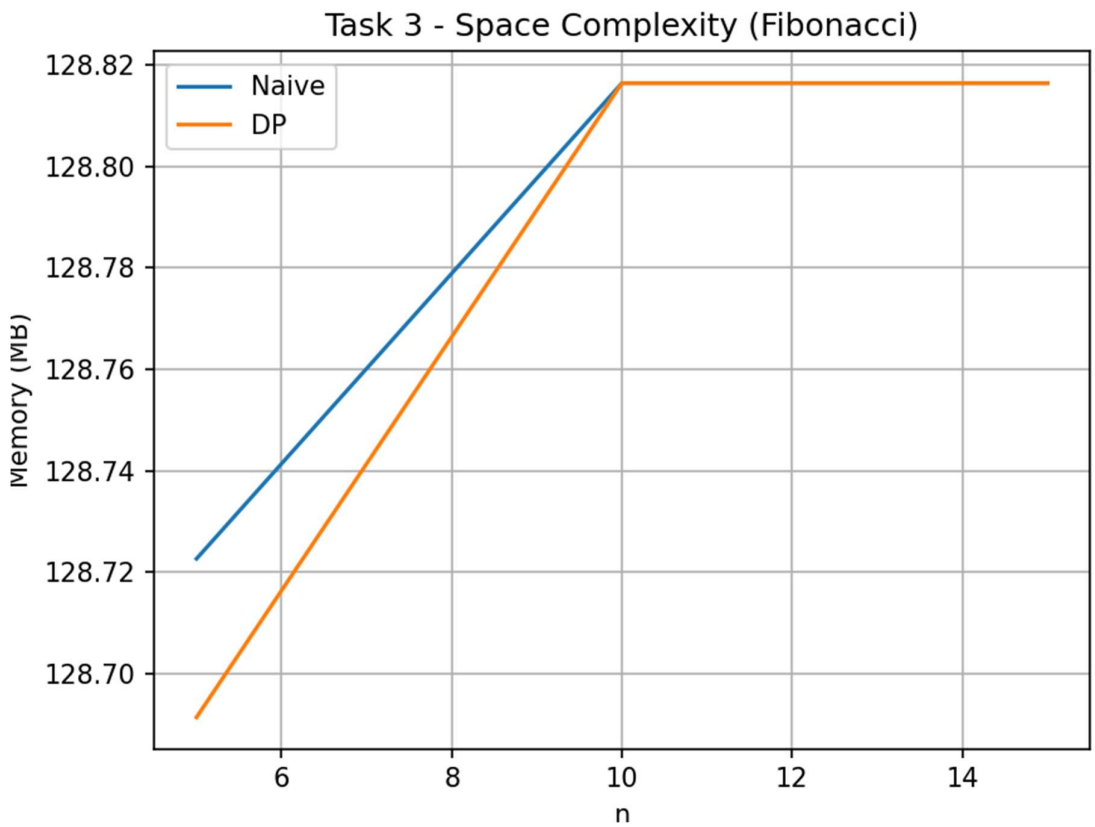
```python
df = pd.DataFrame(results)
print("\nTask 3 Table (Time + Calls):\n")
print(df)
```

```
Task 3 Table (Time + Calls):

                 Algorithm   n              Value  Function Calls  Time (sec)
0  Factorial (Recursion)   5                120               5    0.000003
1  Factorial (Recursion)  10            3628800              10    0.000001
2  Factorial (Recursion)  15         1307674368000              15    0.000002
3  Factorial (Recursion)  20 2432902008176640000              20    0.000002
4      Fibonacci (Naive)   5                  5              15    0.000003
5        Fibonacci (DP)   5                  5               9    0.000003
6      Fibonacci (Naive)  10                 55             177    0.000013
7        Fibonacci (DP)  10                 55              19    0.000005
8      Fibonacci (Naive)  15                610            1973    0.000142
9        Fibonacci (DP)  15                610              29    0.000006
10     Fibonacci (Naive)  20               6765           21891    0.001570
```

```python
fib_df = df[df["Algorithm"].str.contains("Fibonacci")]

plt.figure(figsize=(10, 6))

for algo in fib_df["Algorithm"].unique():
    temp = fib_df[fib_df["Algorithm"] == algo]
    plt.plot(temp["n"], temp["Time (sec)"], marker='o', label=algo)

plt.xlabel("n")
plt.ylabel("Execution Time (seconds)")
plt.title("Fibonacci: Naive Recursion vs DP")
plt.legend()
plt.grid(True)

plt.yscale("log")
plt.show()
```

Result:


Task 3 - Time Complexity (Fibonacci)
Naive
DP
Time (seconds)
18
16
14
12
10
8
6
8
10
12
14


Task 3 - Space Complexity (Fibonacci)
Naive
DP
Memory (MB)
128.82
128.80
128.78
128.76
128.74
128.72
128.70
6
8
10
12
14
n

Task 4: Solving Recurrences Through Code:

```python
import time
import pandas as pd
import matplotlib.pyplot as plt
```

```python
# Recurrence 1:
# T(n) = T(n/2) + n

calls_1 = 0

def T1(n):
    global calls_1
    calls_1 += 1

    if n <= 1:
        return 1
    return T1(n // 2) + n


# Recurrence 2:
# T(n) = 2T(n/2) + n

calls_2 = 0

def T2(n):
    global calls_2
    calls_2 += 1

    if n <= 1:
        return 1
    return 2 * T2(n // 2) + n
```

```python
def measure(func, n):
    start = time.perf_counter()
    val = func(n)
    end = time.perf_counter()
    return val, end - start
```

```python
sizes = [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]

results = []

for n in sizes:
    # T1
    calls_1 = 0
    val1, t1 = measure(T1, n)

    results.append({
        "Recurrence": "T(n) = T(n/2) + n",
        "n": n,
        "Value": val1,
        "Calls": calls_1,
        "Time (sec)": t1
    })

    # T2
```

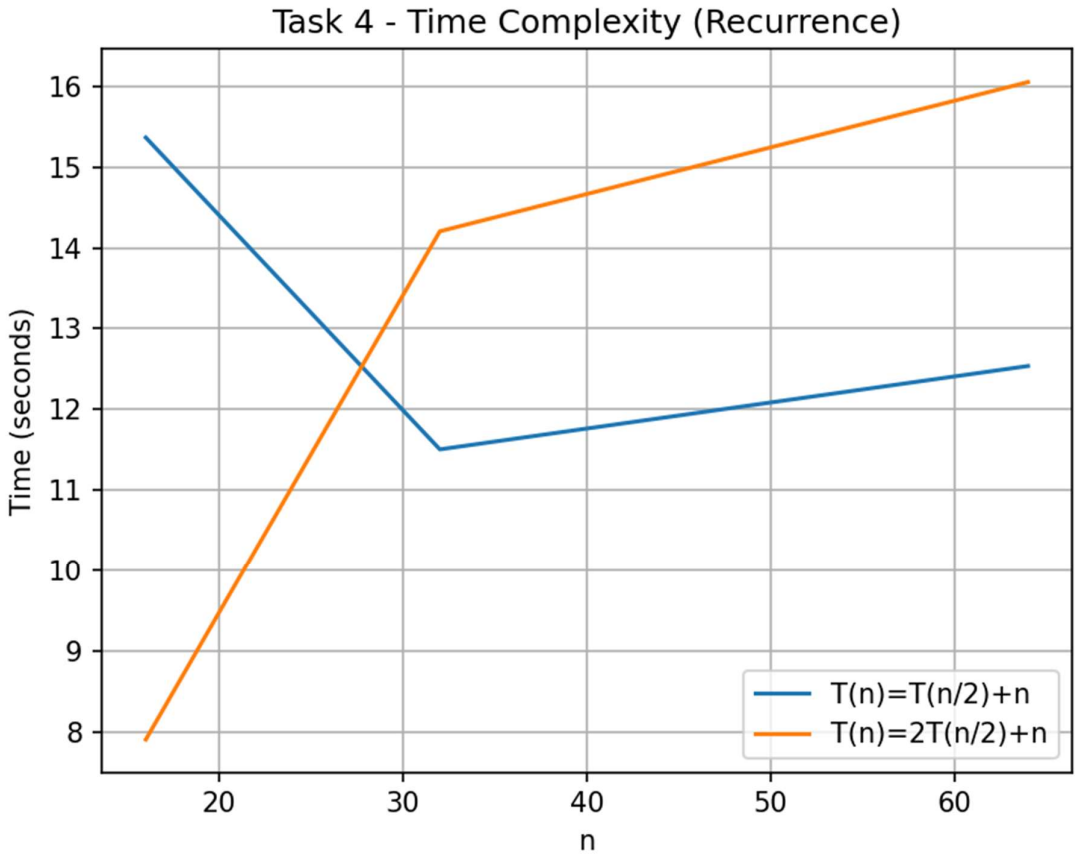
```python
    })

    # T2
    calls_2 = 0
    val2, t2 = measure(T2, n)

    results.append({
        "Recurrence": "T(n) = 2T(n/2) + n",
        "n": n,
        "Value": val2,
        "Calls": calls_2,
        "Time (sec)": t2
    })
```

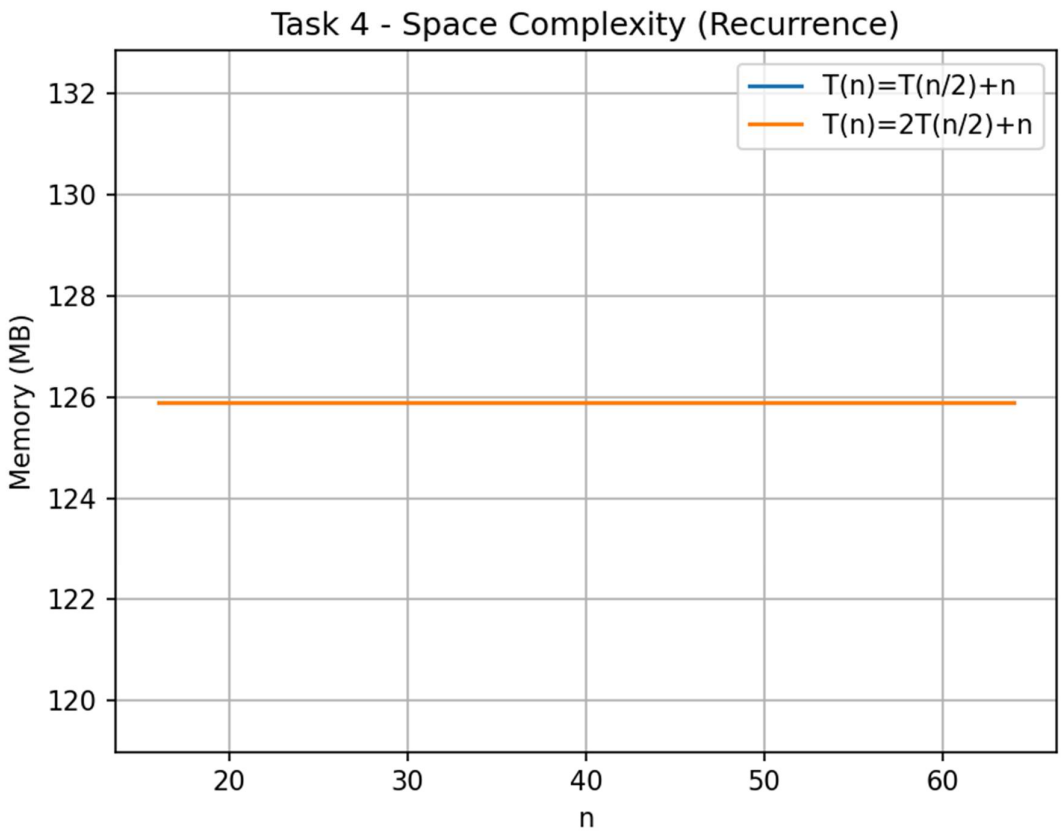
```python
df = pd.DataFrame(results)
print("\nTask 4 Table:\n")
print(df)
```

```
Task 4 Table:

              Recurrence     n  Value  Calls    Time (sec)
0   T(n) = T(n/2) + n     2      3      2  1.490805e-06
1   T(n) = 2T(n/2) + n    2      4      3  1.000000e-06
2   T(n) = T(n/2) + n     4      7      3  7.001217e-07
3   T(n) = 2T(n/2) + n    4     12      3  6.000046e-07
4   T(n) = T(n/2) + n     8     15      4  4.000030e-07
5   T(n) = 2T(n/2) + n    8     32      4  5.001202e-07
6   T(n) = T(n/2) + n    16     31      5  4.990874e-07
7   T(n) = 2T(n/2) + n   16     80      5  4.990874e-07
8   T(n) = T(n/2) + n    32     63      6  6.000046e-07
9   T(n) = 2T(n/2) + n   32    192      6  6.000046e-07
10  T(n) = T(n/2) + n    64    127      7  6.000046e-07
11  T(n) = 2T(n/2) + n   64    448      7  6.990889e-07
12  T(n) = T(n/2) + n   128    255      8  7.001217e-07
13  T(n) = 2T(n/2) + n  128   1024      8  9.990904e-07
14  T(n) = T(n/2) + n   256    511      9  7.001217e-07
15  T(n) = 2T(n/2) + n  256   2304      9  8.000061e-07
16  T(n) = T(n/2) + n   512   1023     10  1.100125e-06
17  T(n) = 2T(n/2) + n  512   5120     10  1.090892e-06
18  T(n) = T(n/2) + n  1024   2047     11  1.200009e-06
19  T(n) = 2T(n/2) + n 1024  11264     11  1.100125e-06
```

```python
plt.figure(figsize=(10, 6))

for rec in df["Recurrence"].unique():
    temp = df[df["Recurrence"] == rec]
    plt.plot(temp["n"], temp["Time (sec)"], marker="o", label=rec)

plt.xlabel("n")
plt.ylabel("Execution Time (seconds)")
plt.title("Recurrence Execution Time Comparison")
plt.legend()
plt.grid(True)

plt.yscale("log")
plt.show()
```

RESULT:


Task 4 - Time Complexity (Recurrence)
Time (seconds)
n
16
15
14
13
12
11
10
9
8
20
30
40
50
60
T(n)=T(n/2)+n
T(n)=2T(n/2)+n



GITHUB REPO LINK:

https://github.com/Code-trac/ada_assignment.git